



AI Agents

The Future of Autonomous AI Systems

Building, Optimizing, and Deploying LLM-Based Agents

Presented by: William Michaud | Date: Jan 2026

What is a LLM Agent?

- ◆ An AI system that uses a language model as its "brain"
- ◆ Plans, executes, and refines actions autonomously
- ◆ Uses tools and external integrations to complete tasks
- ◆ Maintains context across multiple conversation turns
- ◆ Adapts when outcomes don't match expectations

How Does It Work?

Perception: Agent receives user input and context

Planning: LLM breaks down task into steps

Tool Selection: Agent chooses appropriate tools

Execution: Tools execute and return results

Refinement: Agent adapts based on outcomes

But why should we learn to use them?

Because they are **useful**!

Efficiency: Google reported a 25-50% faster completion for many tasks thanks to vibe coding, a developer that can do more will be worth more (and be paid more)!

To stay competitive: Lots of companies are adopting AI everywhere, and they need people who can use them properly!

For the Earth: AI tools consume resources horribly fast, so if you have to, it's better to be able to one-shot a task rather than spending hours debugging with AI...

To learn: AI tools have a lot knowledge that they can teach you, simply by watching how it approaches a complex task or explaining a snippet of code.

Key Features: Tools & Defaults

Default Tools: Built-in capabilities (file I/O, TODO list, math, OCR)

MCP servers: An easy way to customize the toolset.

- ◆ Can be installed locally (in the project folder) or globally, e.g. using npm.
- ◆ Exposes a standardized tool interface that lets models “see” the new tools and use them.

Key Features: Context handling

Context Windows: VERY limited on almost all models.

Models will start hallucinating **aggressively** when the context window is nearly full. Different solutions are available:

Conversation summarizing: Generate a "summary" of the current conversation. Seamless but loss of information with nearly no user control.

Sliding window history: Progressively remove older messages from the history.

Hard Reset: Just start a new conversation with a fresh context window ;)

Key Features: RAG

RAG (Retrieval-Augmented Generation): Fetch & augment context

This is a work-around to increase the information available no matter the context window.

Analogy: It's like giving the LLM an open-book exam instead of forcing it to memorize everything.

We "index" the codebase, and let the agent search this index by meaning instead of text: the agent can "understand" the entire codebase even with a small context window.

Key Features: Tasks

Background Tasks: Async execution & cloud integration

It can often be convenient to be able to ask the agent to start a long task, leave/close your computer and come back when it's done ;)

Key Features: Version Control

Git Support: Commit comment & review

An AI-generated message could explain a commit better than you, to let you focus only on the code!

Pull requests:

Some agents are also able to review pull requests, to help developer understand what was changed much quicker.

Key Features: Compatibility

Model Compatibility: Works across different LLMs

- ◆ Most LLM companies have their own “AI agent” client, but it often has limited functionalities and only supports their own models.

Examples & Landscape

Example: GitHub Copilot

The Industry Standard

Why you should use it:

Deeply integrated into VS Code & GitHub. Great for autocompletion and basic chat. It's the "Apple" of AI agents—it just works, but customizability can be limited compared to open-source alternatives.

Example: KiloCode AI / Open Alternatives

The Power User Choice

Why you should use it:

Open-source platforms often offer better context management (like sliding windows or manual control) and "Agent Mode" where the AI can take control of your editor to perform multi-step refactoring tasks autonomously.

How to Choose a LLM Model

Rankings: Benchmark results and users usage

Cost Analysis: Price per token vs. output quality

Speed: Latency and throughput (tokens/sec)

It is ALWAYS changing, don't believe that a model will be the best forever!

OpenRouter: Model Hub & Comparison

Unified Access: 200+ models in one platform

Model Search tools: Multiple filters to list and pick the best model for a task

Fallback Routing: Automatically switch models if one is unavailable

Rankings: Regular rankings based on different criteria

Prompt Engineering Fundamentals

Here are a few tips for improving your prompts!

Clarity: Be specific and unambiguous, no "fix this pls"

Task planning: Work sequentially

Ask the model to only do a small part of the work, and then improve over it iteratively to give it opportunities to fix the mistakes.

Context: Provide relevant information upfront

Keep track of what the model currently knows, what it has access to, and what it can't access to avoid random guesses.

Over-Refining Trap: Avoid micro-optimizing endlessly

Models will start to forget / ignore information if the prompt is too long or detailed!

Prompt Auto-Refine

Auto-Refine: Let agent improve its own prompts

Most agent extensions have an “auto-refine” or “auto-improve” button for prompts.

Sometimes, it is also done by default on each prompt before the agent starts interpreting it.

The idea is that each model will understand a concept in a slightly different way, so by simply asking the same model to reformulate your prompt, you can ensure that you both understood the same thing.

Effective Task Planning

Decomposition: Break complex goals into steps

Sequencing: Order steps for optimal execution

Dependencies: Track task prerequisites

Checkpoints: Verify progress at key stages

Context Enrichment

Here are a few tricks to optimize the knowledge and behavior of your agent:

Context Enrichment: Link relevant files to your request, avoid copy pasting without context.

System Prompts: Foundation for agent behavior to add rules and behavior that should always be followed.

Agent Modes: Specialized behaviors and tool sets for different tasks.

Rule Files

Rule Definition: Hard constraints on behavior

They can be defined on the project level (current repo) or globally (entire computer)

This is an extension of the system prompt, usually with more flexibility

Usage examples:

- ◆ Only use Windows-style CLI syntax
- ◆ Read the README to have a high-level understanding of the project
- ◆ Do not try to modify / delete this file: ...

Application: One-Shot Snake Bot

Quick challenge: Let's create a playable Snake game AI with a single prompt!

Let's copy-paste a prompt into the agent and see what happens!